

Research Walkthrough

Grant Seeker AI - Architecture Research Walkthrough

I have completed a deep-dive analysis of the **Grant Seeker AI** project architecture and technical details. This project is a sophisticated multi-agent system designed to automate the grant-seeking lifecycle, with a specific focus on the Canadian non-profit sector.

? Key Findings

1. **Core Technology Stack**

The project is built using a modern AI-centric stack:

- * **Framework**: Python with **Google Agent Development Kit (ADK)** for agent orchestration.
- * **AI Model**: **Google Gemini 1.5 Flash** (leveraging its speed and cost-efficiency).
- * **Frontend**: **Streamlit** (multi-page application with custom CSS).
- * **Search & Scraping**: **Tavily API** (AI-specialized search) with robust fallbacks to Google Search, PDF parsing, and BeautifulSoup.

2. **Multi-Agent Architecture**

The system uses a **Hybrid Multi-Agent** approach where specialized agents collaborate:

- * **QueryGenerator**: Optimizes user input for Canadian grant searches.
- * **GrantFinder**: Vets search results to find high-quality leads.
- * **GrantExtractor**: Pulls structured data (deadlines, amounts, eligibility) from diverse web content.
- * **GrantWriter**: Drafts persuasive proposal sections tailored to specific grants.

3. **Engineering Highlights**

- * **Robust Extraction**: A tiered extraction system (`content_extractor.py`) ensures that "Untitled Grant" errors are minimized by falling back through multiple scraping strategies.
- * **Iterative Search**: A "Search Thoroughness" feature that keeps searching until a minimum number of relevant results are found.
- * **Smart Caching**: A file-based caching system (`.cache/`) that drastically reduces latency and API costs.
- * **Pydantic Validation**: All data flow between agents and the UI is strictly typed and validated, ensuring system stability.

? Artifacts Created

I have documented the full technical details in the following artifact:

*

[architecture_overview.md](file:///Users/daai6ga1hou2/.gemini/antigravity/brain/268ebebc-4c43-4166-89ae-e7b3375945b3/architecture_overview.md) - A comprehensive technical guide including a Mermaid architecture diagram.

? Verification Results

* **Files Analyzed**: `backend/adk\_agent.py`, `backend/content\_extractor.py`, `frontend/search\_grants.py`, `home\_page.py`, etc.

* **Components Verified**: Agent prompting, caching logic, scraping strategies, and Streamlit navigation flow.

* **Architecture Mapping**: Successfully mapped the end-to-end data flow from user input to proposal drafting.